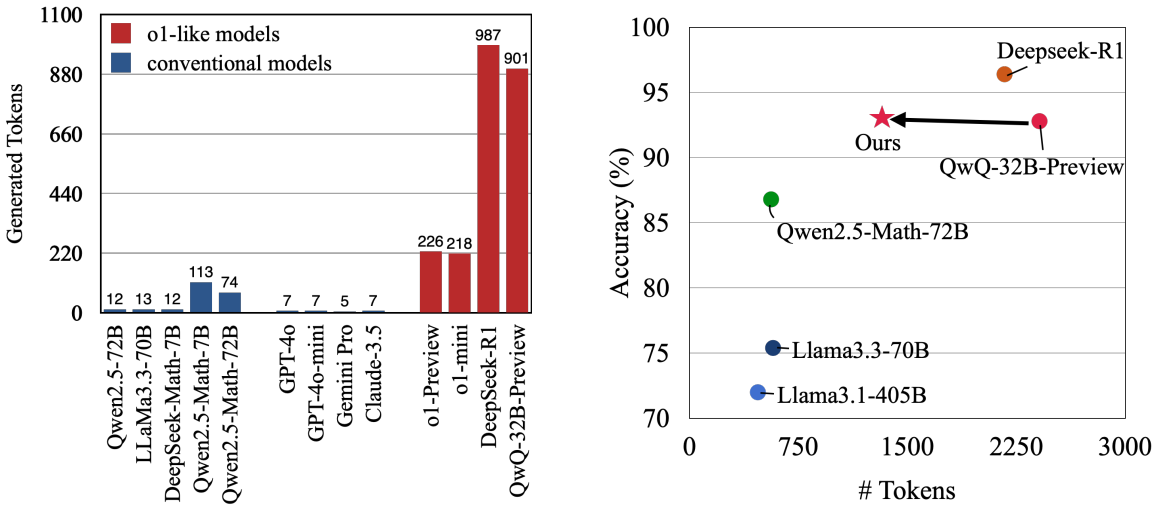
Do NOT Think That Much for 2+3=? On the Overthinking of o1-Like LLMs

Xingyu Chen[*,1,2] **, Jiahao Xu**[*,1] **, Tian Liang**[*,1] **, Zhiwei He**[*,1,2] **, Jianhui Pang**[1] **, Dian Yu**[1] **, Linfeng Song**[1] **, Qiuzhi Liu**[1] **, Mengfei Zhou**[2] **, Zhuosheng Zhang**[2] **, Rui Wang**[†2] **, Zhaopeng Tu**[†1] **, Haitao Mi**[1] **, and Dong Yu**[1]

[1]Tencent AI Lab
[2]Shanghai Jiao Tong University



(a) Generated tokens on question "what is the answer of 2 plus 3?"

(b) Token-accuracy plot on MATH500

Figure 1: Illustration of **overthinking issue** in Figure (a): o1-like models (right panel) spend much more tokens than conventional LLMs (left and middle panels). Our method reduces the overthinking issue when applied to QwQ-32B-Preview (Figure (b)).

Abstract

The remarkable performance of models like the OpenAI o1 can be attributed to their ability to emulate human-like long-time thinking during inference. These models employ extended chain-of-thought (CoT) processes, exploring multiple strategies to enhance problem-solving capabilities. However, a critical question remains: *How to intelligently and efficiently scale computational resources during testing*. This paper presents the first comprehensive study on the prevalent issue of **overthinking** in these models, where excessive computational resources are allocated for simple problems with minimal benefit. We introduce novel efficiency metrics from both outcome and process perspectives to evaluate the rational use of computational resources by o1-like models. Using a self-training paradigm, we propose strategies to mitigate overthinking, streamlining reasoning processes without compromising accuracy. Experimental results show that our approach successfully reduces computational overhead while preserving model performance across a range of testsets with varying difficulty levels, such as GSM8K, MATH500, GPQA, and AIME.

[*]Equal Contribution. The work was done when Xingyu and Zhiwei were interning at Tencent AI Lab.
[†]Correspondence to: Zhaopeng Tu <zptu@tencent.com> and Rui Wang <wangrui12@sjtu.edu.cn>.

1 Introduction

The OpenAI o1 model (OpenAI, 2024) and its replicas (Qwen, 2024; Guo et al., 2025; Kimi et al., 2025) exemplify the state-of-the-art in AI reasoning. Their success is largely attributed to mimicking human-like long-time thinking before responding to a question. Specifically, o1-like models cultivate a long chain-of-thoughts (CoT), explore multiple strategies, break down complex steps, and perform double-checking, which ultimately enhance their ability to tackle intricate reasoning tasks. This approach, known as "scaling test-time compute", involves allocating more computational resources during the model's inference phase to generally yield more accurate responses.

While effective, a critical yet underexplored question remains: **Are we scaling test-time compute efficiently and intelligently?** This study provides an initial exploration of this problem. We first observe that o1-like models exhibit significant **overthinking** issues. Specifically, they tend to expend excessive compute (in terms of tokens or thinking rounds) on questions that are exceptionally simple or for which the answer is already evident. For example, Figure 1(a) compares the token usage of o1-like models with conventional models when answering the question, "what is the answer of 2 plus 3?" On average, o1-like models consumed **1,953%** more tokens than conventional models to reach the same answer. Figure 2 illustrates a concrete example where o1-style thinking results in generating 13 solutions for this trivially simple question. Across extensive analyses of mathematical benchmarks, we found these overthinking patterns: (1) contribute minimally to improving accuracy, (2) lack diversity in reasoning strategies, and (3) occur more frequently with simple problems.

The overthinking observed in o1-like models reveals inefficiency in inference and highlights fundamental limitations in their reasoning and decision-making processes. We assert that reasoning involves not only accuracy but also the application of the appropriate level of complexity based on the problem's requirements. This insight motivates our exploration of studying and mitigating overthinking. To address this, we propose two metrics from both outcome and process perspectives to evaluate o1-like models' efficiency. These metrics help provide a comprehensive assessment of the **efficiency** of o1-like models, augmenting the commonly-used **effectiveness** metrics.

To mitigate overthinking without introducing external information, we adopt a self-training paradigm. With our proposed efficiency metrics, we streamline the generated responses by removing redundant solutions while maintaining basic reflexivity. Experimental results across testsets of varying difficulty levels (e.g., GSM8K, MATH500, GPQA, and AIME) demonstrate our approach's effectiveness and robustness in mitigating overthinking issues. For instance, as shown in Figure 1(b), our approach can reduce token output by 48.6% while maintaining accuracy on the widely-used MATH500 testset as applied to QwQ-32B-Preview.

In summary, our contributions are three-fold:

1. We present the first study offering both a definitive explanation and comprehensive analysis of the overthinking issue, showing that o1-like LLMs often expend unnecessary computational resources on redundant solutions that contribute minimally to final outcomes.
2. We introduce metrics considering both outcome and process aspects to assess the efficiency of o1-like models.
3. We explore several strategies to mitigate overthinking, significantly reducing token generation while maintaining model performance across testsets of varying difficulty.

2 Observing Overthinking Issues

In this section, we present a comprehensive analysis of outputs generated by o1-like models. First, we provide a basic illustration of the solution distribution in responses from these models (§ 2.1). We then identify two inefficiencies in long CoT responses: their limited contribution to accuracy (§ 2.2) and diversity (§ 2.3). To evaluate these inefficiencies empirically, we propose two efficiency metrics based on our observations. Finally, we present empirical results in § 2.4 and conclude that *o1-like models often overthink, particularly with easier math problems*.

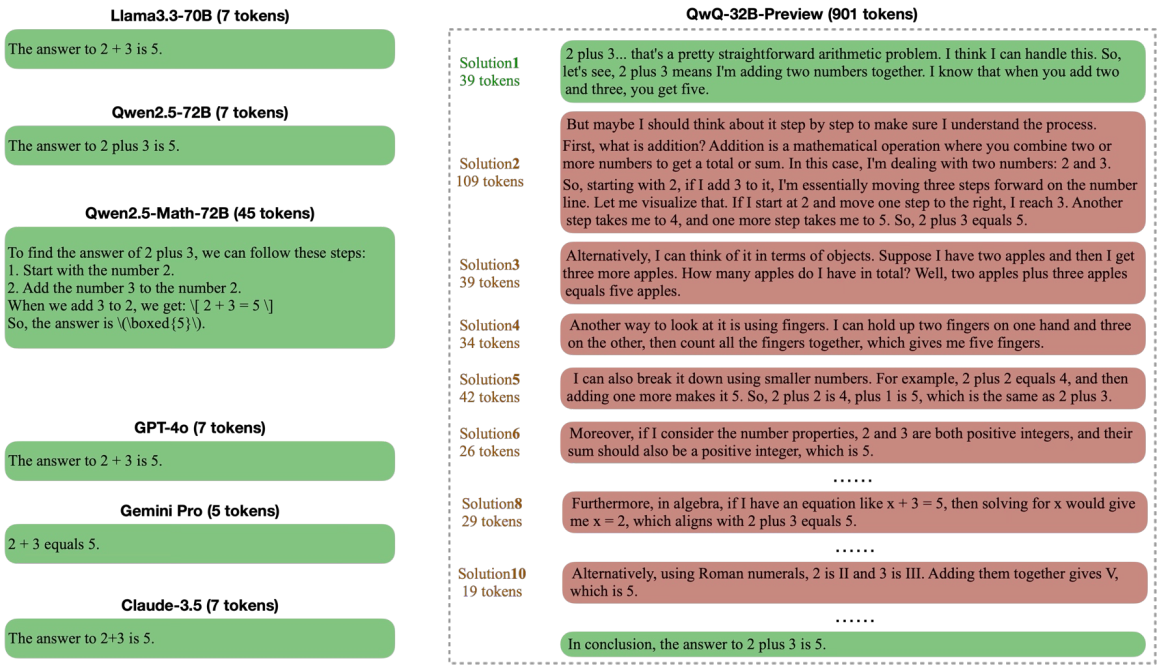


Figure 2: An example of overthinking issue for QwQ-32B-Preview model's output response that consists of 13 solutions. We also list the outputs of other conventional LLMs for reference.

2.1 Solution Distribution of o1-Like Models

Experimental Setup We conduct experiments on three testsets:

- **ASDIV** (Miao et al., 2020): an English math word problem corpus with 2,305 instances, each annotated with its problem type and grade level (1 to 6, indicating difficulty). The test set covers three main problem types (i.e., *basic arithmetic operations*, *aggregative operations*, and *additional domain knowledge required*), typically found in **elementary schools**.
- **GSM8K** (Cobbe et al., 2021): a dataset of high-quality, linguistically diverse **grade school math word problems** created by human problem writers. The test set includes 1,319 problems, with solutions often involving a sequence of elementary calculations using basic arithmetic. A middle school student should be able to solve every problem.
- **MATH500** (Hendrycks et al., 2021): a challenging dataset consisting of problems from **high school math competitions** across seven subjects (e.g., Prealgebra, Algebra, Number Theory) and difficulty levels based on AoPS (ranging from 1 to 5). Problems in these competitions range from level 1, the easiest, often found in AMC 8 exams, to level 5, like those in AIME.

The overall difficulty levels of the test sets are ASDIV $<$ GSM8K $<$ MATH500.

We mainly investigate two widely recognized o1-like models featuring a visible thinking process: Qwen-QwQ-32B-Preview (Qwen, 2024) and DeepSeek-R1 (DeepSeek, 2025).

Solution Distribution In this paper, we define *solution* as part of the full model generation that contains an answer explicitly. For example, in Figure 2, each solution in the QwQ generation contains the answer 5. We use the Llama-3.3-70B model to separate solutions from generated responses. Figure 3 shows the distribution of solutions in generated responses. Generally, o1-like models produce 2 to 4 solution rounds for most instances, covering 76% to 80% of cases for QwQ-32B-Preview across the test sets and 59% to 63% for DeepSeek-R1. Regarding different test sets, o1-like models tend to generate more solutions for easier test sets. For instance, the average number of

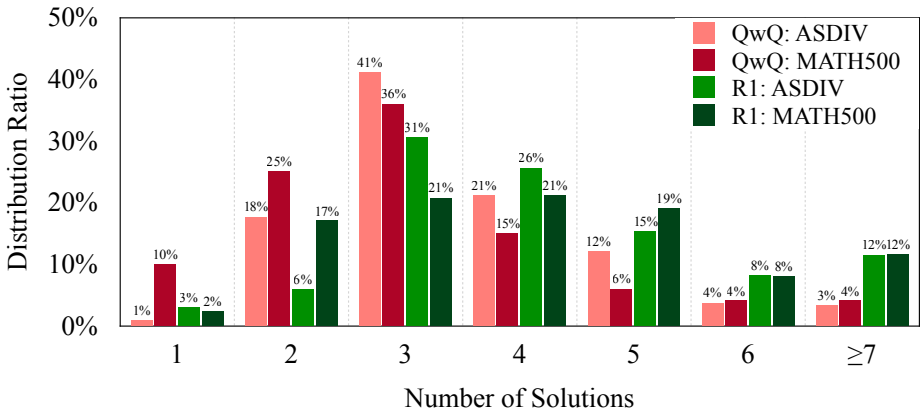
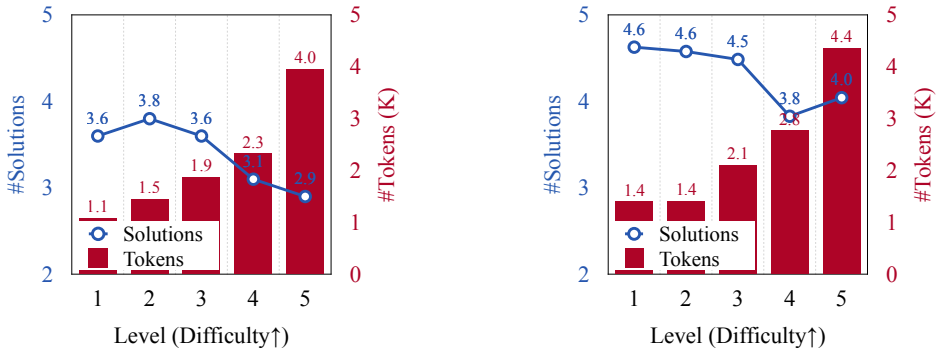


Figure 3: Distribution of solution counts in generated responses for different test sets and models (QwQ-32B-Preview ("QwQ") and DeepSeek-R1 ("R1")).

solutions of QwQ-32B-Preview on the easiest ASDIV test set is 3.5, whereas on the most difficult MATH500 test set, it is 3.2. The numbers for DeepSeek-R1 are respectively 4.5 and 4.3.



(a) QwQ-32B-Preview (b) DeepSeek-R1

Figure 4: Average rounds of solutions ("Solutions") and number of tokens ("Tokens") in generated responses across different difficulty levels of the MATH500 test set.

To empirically validate this finding, we conducted an analysis across various difficulty levels in the MATH500 test set, as illustrated in Figure 4. Both QwQ-32B-Preview and DeepSeek-R1 generate more solution rounds for problems at easier levels 1-2 (e.g., averaging 3.7 rounds and 4.6 rounds, respectively) compared to levels 4-5 (e.g., averaging 3.0 rounds and 3.9 rounds, respectively), despite the number of tokens consistently increasing with the difficulty level. These results support our claim that **o1-like models tend to generate more solution rounds for easier math problems**.

2.2 Efficiency on Accuracy Improvements

Intuition In the example in Figure 2, we observe that the initial round of solutions already yields the correct answer. Subsequent solutions, which account for the majority of generated tokens, do not enhance accuracy. Based on this observation, we empirically investigate whether later solutions contribute to accuracy improvements. Specifically, for all cases where o1-like models produce the correct answer in the response, we calculate the distribution of occurrences for the first correct answer, termed the "first correctness distribution". If more correct answers appear in earlier

solutions, then the subsequent solutions contribute minimally to accuracy improvement, indicating reduced efficiency.

Observation Figure 5 illustrates the first correctness distribution across the test sets and models. In more than 92% of cases, the initial round of solutions produces the correct answer. Notably, the first round generally comprises less than 60% of the total tokens generated, suggesting that the extended CoT might not significantly enhance accuracy. For instance, the average length of the first round of solutions for QwQ-32B-Preview on the ASDIV test set is 287 tokens, constituting only 38.7% of the entire response. These results suggest that **later solutions marginally contribute to improvements in accuracy**.

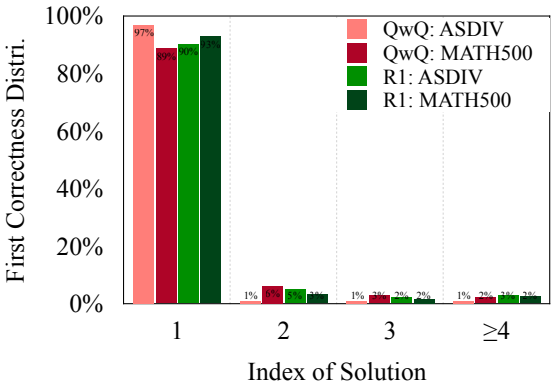


Figure 5: Distribution of occurrences for the first correct answer.

Outcome Efficiency Metric Based on the above observation, we propose an outcome efficiency metric to empirically evaluate how effectively later solutions contribute to accuracy improvements. The outcome efficiency metric, denoted ξ_O, is defined by the following formula:

$$\xi_O = \frac{1}{N}\sum_{i=1}^{N}\sigma_i\frac{\hat{T}_i}{T_i} \tag{1}$$

where N is the number of instances in a given test set, T_i is the total number of tokens produced for the i-th instance, and $\hat{T}_i$ denotes the efficient tokens that contribute to reaching the correct answer:

$$\hat{T}_i = \begin{cases} \text{\#tokens to first arrive at correct answer}, & \sigma_i = 1 \\ T_i, & \sigma_i = 0 \end{cases}$$

σ_i denotes whether the evaluated model can produce a correct answer in the response:

$$\sigma_i = \begin{cases} 1, & \text{if at least one solution in response is correct} \\ 0, & \text{otherwise} \end{cases}$$

Intuitively, if a model correctly answers at an early stage, the tokens generated thereafter do not contribute to improving accuracy and are considered inefficient. Consider Figure 2 as an example: The first solution correctly addresses the problem with $\hat{T} = 39$. Consequently, $\xi_O = \frac{39}{901} = 4.3\%$, which can be considered extremely inefficient.

2.3 Efficiency on Diverse Thinking

Intuition Some researchers might argue that while solving an easy math problem may appear straightforward, approaching it from different perspectives can deepen understanding and build flexibility in mathematical thinking, which is also valuable. Consider the example output of QwQ-32B-Preview in Figure 2: Solution 1 states the basic fact that 2 plus 3 equals 5; Solution 2 breaks the addition into smaller steps; Solution 3 uses a counting objects analogy. These three solutions provide different reasoning strategies. However, Solution 4 repeats Solution 3, and Solution 5 repeats Solution 2 using similar reasoning strategies. In this section, we empirically examine the diversity among solutions within a response.

Observation To empirically evaluate whether later solutions provide new reasoning strategies, we introduce the "distinctness ratio" as the measure for the ratio of distinct solutions for each data index. Consider $R_i = \{s_i^1, \ldots, s_i^m, \ldots, s_i^{M_i}\}$ as the set of M_i solutions in the i-th instance response.

Let $S^m = \{s_1^m, \ldots, s_k^m, \ldots, s_K^m\}$ be the set of m-th solutions in the responses of all instances in the test subset.[1] The distinctness ratio is defined as:

$$\text{Dis}^m = \frac{\sum_{k=1}^{K} \tau_k^m}{K}$$

where

$$\tau_k^m = \begin{cases} 1, & \text{if } \Phi(s_k^m) \notin \{\Phi(s_k^1), \ldots, \Phi(s_k^{m-1})\} \\ 0, & \text{otherwise} \end{cases}$$

In this context, $\Phi(s_k^m)$ is the reasoning strategy of s_k^m. We use GPT-4o to cluster the solutions for each instance into groups via a prompt like (Ye et al., 2024).[2] The clustering results for the QwQ-32B-Preview response in Figure 2 are:

cluster1 [Solution 1, Solution 6, Solution 11] stating or affirming the basic arithmetic fact that 2 plus 3 equals 5.
cluster2 [Solution 2, Solution5] breaking down the addition into smaller, simpler steps to reach the result.
cluster3 [Solution 3, Solution 4] using a practical analogy of counting objects to explain the addition.
cluster4 [Solution 7] using subtraction as a reverse check to verify the addition result.
cluster5 [Solution 8] using algebraic manipulation and solving simple equations to confirm the result.
cluster6 [Solution 9, Solution 10] converting numbers into different systems (binary and Roman numerals) to verify the result.
cluster7 [Solution 12, Solution 13] considering specific contexts or frameworks like modular arithmetic or programming which could change traditional addition results.

Figure 6 displays the distinctness ratio for each solution index. Intuitively, the ratio for Solution#1 is always 100%, as it has no preceding solutions, thus $\tau \equiv 1$ for all instances. Generally, the ratio decreases with higher indices, indicating that **later solutions often repeat earlier ones**. For example, the average distinctness ratio for Solution#≥4 across test sets decreases by 11.5% compared to Solution#3. The ratio of Solution#2 significantly decreases, underperforming Solution#3. By reviewing outputs, we find that Solution#2 often double-checks answers from Solution#1 using the same reasoning strategy. Subsequently, Solution#3 attempts to solve the problem using a new reasoning strategy.

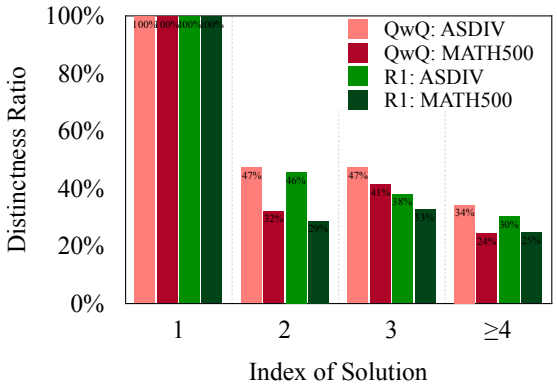


Figure 6: Ratio of whether a solution provides a new reasoning strategy for each index.

Process Efficiency Metric Based on the above observation, we propose a process efficiency metric to empirically evaluate the contribution of later solutions to solution diversity. The process efficiency metric, denoted ξ_P, is calculated using the formula:

$$\xi_P = \frac{1}{N} \sum_{i=1}^{N} \frac{D_i}{T_i} \tag{2}$$

[1] If a response does not contain the m-th solution (i.e. $M_i < m$), that response is excluded from the set, hence K does not necessarily equal the number of test set instances N.

[2] Refer to Appendix A.2 for clustering prompt details.

Table 1: Model efficiency results of strong LLMs.

Models	Accuracy	Response		Efficiency	
		#Solution	#Token	Outcome	Process
ASDIV					
Llama-3.3-70B-Instruct	95.6	1.0	166.4	95.6%	100.0%
Qwen2.5-Math-72B-Instruct	96.3	1.0	213.0	96.3%	100.0%
QwQ-32B-Preview	96.9	3.5	741.8	41.9%	66.5%
DeepSeek-R1	97.1	4.5	845.0	45.9 %	64.3%
GSM8K					
Llama-3.3-70B-Instruct	92.6	1.0	220.3	92.6%	100.0%
Qwen2.5-Math-72B-Instruct	95.8	1.0	317.4	95.8%	100.0%
QwQ-32B-Preview	94.8	3.1	772.8	50.7%	67.6%
DeepSeek-R1	96.4	4.3	1056.3	48.9%	62.0%
MATH500					
Llama-3.3-70B-Instruct	75.4	1.0	553.4	75.4%	100.0%
Qwen2.5-Math-72B-Instruct	86.8	1.0	593.1	86.8%	100.0%
QwQ-32B-Preview	93.0	3.2	2407.9	52.3%	71.2%
DeepSeek-R1	96.4	4.3	2704.3	51.0%	66.2%

where D_i represents the number of efficient tokens that contribute to the solutions' diversity. Here, we intentionally exclude the factor σ_i to concentrate on diversity, **independent of correctness**. Let T_i^m denote the number of tokens in solution s_i^m. We define:

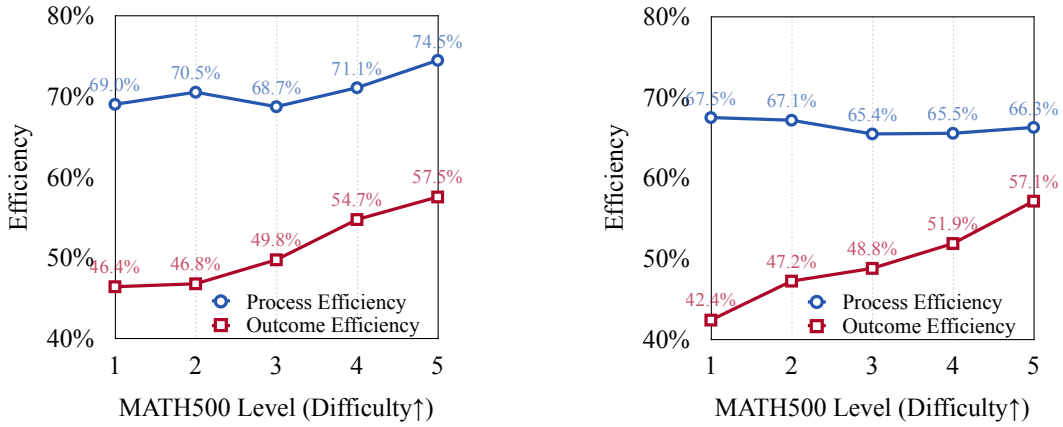
$$D_i = \sum_{m=1}^{M} \tau_i^m T_i^m$$

Intuitively, the tokens in a distinct solution are regarded as process efficient tokens. In the example shown in Figure 2, the 13 solutions are categorized into 7 distinct reasoning strategies. Consequently, tokens in Solutions 1, 2, 3, 7, 8, 9, and 12 are efficient, resulting in $\xi_P = \frac{(39+109+39+29+29+19+59)}{901} = 35.8\%$.

2.4 Empirical Efficiency Results

Table 1 presents the results on model efficiency. For comparison, we include two representative conventional LLMs: Llama-3.3-70B-Instruct and Qwen2.5-Math-72B-Instruct. These conventional LLMs produce only a single solution, meaning that $\frac{D_i}{T_i} = \frac{\hat{T}_i}{T_i} = 1$. Therefore, in these cases, the outcome efficiency metric $\xi_O = \frac{1}{N}\sum_{i=1}^{N} \sigma_i$ equals accuracy, and the process efficiency metric $\xi_P = 1.0$. In comparison, o1-like models generate significantly longer responses, which are less efficient in improving accuracy and solution diversity. We refer to the inefficient use of generated tokens as the "**overthinking issue**". The experimental results demonstrate that while o1-like models have the capacity to generate multiple solutions, their efficiency is hindered by the overthinking issue.

Figure 7 presents the detailed efficiency results across various difficulty levels of the MATH500 test set. Notably, both models perform poorly on the simplest Level 1 problems, achieving less than 50% outcome efficiency, a pattern that corresponds with results observed on the easy ASDIV test set. These findings underscore that **the overthinking issues faced by o1-like models are particularly pronounced with simpler math problems**.



(a) QwQ-32B-Preview (b) DeepSeek-R1

Figure 7: Efficiency results of (a) QwQ-32B-Preview and (b) DeepSeek-R1 across different difficulty levels of the MATH500 testset.

Table 2: Statistics on different types of generated responses based on the training data. "Greedy" denotes responses generated via greedy search; "Shortest" and "Longest" indicate the shortest and longest responses among 10 samples, respectively.

Response	#Solution	#Token	Efficiency	
			Outcome	Process
Greedy	3.1	1434.8	55.6%	72.6%
Shortest	2.5	1051.3	69.8%	80.3%
Longest	4.1	2258.7	46.0%	66.4%

3 Mitigating Overthinking Issues

In this section, we explore several strategies aimed at enhancing the efficiency of o1-like models. We adopt the settings for LLM reasoning tasks and primarily utilize the self-training strategy (Zelikman et al., 2022; Ho et al., 2023), where the model itself generates the training data. Consistent with previous studies, we employ the PRM12K dataset (Lightman et al., 2024) as our training dataset to generate self-training data. The QwQ-32B-Preview model serves as our testing platform because it is available for post-training.

3.1 Length Preference Optimization

We began by assessing whether the model could produce more efficient responses. We generated 10 samples for each instance in the training dataset with a temperature of 1.0. We discard samples that failed to generate a correct answer. Table 2 presents the statistics of different types of generated responses. Our analysis of these sampled responses reveals that the shortest response performs better in terms of both outcome and process efficiency, using fewer rounds and tokens. These findings support our initiative to enhance model efficiency through self-improvement.

We explore several effective methods for self-improvement:

- **Supervised Fine-Tuning** (SFT; Wei et al. 2022a): This method involves fine-tuning a model using positive synthetic data. The model learns to map inputs to preferred outputs by minimizing the cross-entropy loss between predicted and actual outputs. SFT enables the model to mimic the behavior demonstrated in training examples.

Table 3: Statistics on different types of positive examples. "#S" denotes the number of solutions.

Positive Example	#S	#Token	Efficiency	
			Outcome	Process
Shortest Response	2.5	1051.3	69.8%	80.3%
FCS	1.1	681.0	99.5%	99.1%
FCS + Ref.	1.9	878.7	78.4%	82.4%
GDS	1.6	856.8	86.8%	94.2%

- **Direct Preference Optimization** (DPO; Rafailov et al. 2024): This method trains a model directly on human-preferred responses to increase the likelihood of preferred responses over unpreferred ones.
- **Reasoning Preference Optimization** (RPO; Pang et al. 2024; Liu et al. 2024): This approach modifies the DPO loss by adding a negative log-likelihood term on the preferred response. RPO enhances DPO training stability by preventing a decreased probability of selected responses.
- **Simple Preference Optimization** (SimPO; Meng et al. 2024): This method addresses the discrepancy between the reward function and the generation metric during inference found in other preference optimization methods. SimPO incorporates techniques like adaptive margin and length regularization into DPO training.

Apart from the SFT method, which uses only the shortest sampled response as training data, the other three preference optimization methods require contrastive instance pairs (positive, negative). It is straightforward to use the response generated by greedy search as the negative example, aligning with the real-time inference scenario. However, in our preliminary experiments, we found it less effective than using the longest sampled response as the negative example. One possible reason is that the longest sampled response provides a clearer contrastive signal.

3.2 Streamlining Responses to Enhance Efficiency

Although shorter sampled responses improve the efficiency of o1-like models, they still suffer from overthinking issues. Based on the observations in Section 2, where earlier solutions in the response are more efficient, we further streamline the responses to enhance efficiency. We propose three types of simplification strategies that differ in how they streamline the responses from the beginning:

- **First-Correct Solutions (FCS)**: This strategy retains the earliest solutions that first arrive at the correct answer.
- **FCS+Reflection**: Since the majority of responses achieve the correct answer on the first solution (see Figure 5), maintaining only the First-Correct Solutions might cause o1-like models to revert to conventional LLM behavior. To counter this, we extend the approach to include the second solution that reaches the correct answer in positive examples, recalling the model's long-reflective capability while maintaining efficiency.
- **Greedily Diverse Solutions (GDS)**: Figure 6 demonstrates that the distinctiveness of Solution#2 significantly decreases because the second solution often double-checks answers from the first using the same reasoning strategy. Consequently, FCS+Reflection may reduce efficiency. To address this issue, we propose a simple heuristic that greedily expands solutions providing new reasoning strategies. Additionally, this strategy includes more solutions when the second solution does not repeat the first, thereby increasing diversity.

For each instance, we select the shortest result of each type from 10 samples. Consequently, the three types of simplified responses may originate from different original responses. Table 3 presents the statistics for these simplified responses. Notably, all simplified responses enhance efficiency compared to the shortest response. "FCS" is the most efficient, both in terms of outcome and process,

Table 4: Experimental results of the proposed efficiency enhancing methods.

Methods	Accuracy	Response		Efficiency	
		#Solution	#Token	Outcome	Process
		ASDIV			
QwQ-32B-Preview	96.9	3.5	741.8	41.9%	66.5%
+SimPO$_{\text{FCS+Reflection}}$	96.8	2.0	381.6	77.6%	86.0%
		GSM8K			
QwQ-32B-Preview	94.8	3.1	772.8	50.7%	67.6%
+SimPO$_{\text{FCS+Reflection}}$	96.0	2.0	416.6	80.2%	87.2%
		MATH500			
QwQ-32B-Preview	93.0	3.2	2407.9	52.3%	71.2%
+SFT$_{\text{Shortest Response}}$	93.2	3.0	2359.5	60.4%	75.6%
+DPO$_{\text{Shortest Response}}$	94.0	2.7	1929.5	65.8%	79.1%
+RPO$_{\text{Shortest Response}}$	91.6	2.7	2015.7	64.8%	79.2%
+SimPO$_{\text{Shortest Response}}$	92.4	2.5	1871.8	67.6%	80.9%
+SimPO$_{\text{First-Correct Solution}}$	91.0	1.4	1016.0	88.7%	98.1%
+SimPO$_{\text{FCS+Reflection}}$ (Ours)	92.8	1.9	1330.7	80.0%	89.5%
+SimPO$_{\text{Greedily Diverse Solutions}}$	91.8	1.7	1286.1	84.3%	93.6%
		GPQA			
Qwen2.5-Math-72B-Instruct	46.5	1.0	811.7	46.5%	100%
QwQ-32B-Preview	59.6	2.2	3228.4	51.4%	84.3%
+SimPO$_{\text{FCS+Reflection}}$	59.1	1.7	2085.7	55.7%	90.4%
		AIME24			
Qwen2.5-Math-72B-Instruct	23.3	1.0	1204.5	23.3%	100.0%
QwQ-32B-Preview	46.7	2.6	9480.9	38.4%	84.4%
+SimPO$_{\text{FCS+Reflection}}$	43.3	1.7	5154.5	39.8%	92.0%

using the fewest number of solution rounds and tokens. "FCS+Reflection" incorporates reflection, requiring approximately one additional solution round, which reduces both outcome and process efficiencies. "Greedily Diverse Solutions" serves as a compromise, balancing the number of solutions and tokens, and achieving moderate to high efficiency.

3.3 Experimental Results

Table 4 presents the results of the proposed methods. We perform a detailed comparison on MATH500 and validate the most effective approach using the other test sets.

Performance of Length Preference Optimization Methods SFT only slightly reduces the number of solution rounds and tokens compared to the vanilla QwQ-32B-Preview model, underperforming the preference optimization methods. Among these methods, SimPO achieves the best results, reducing the number of generated tokens by 22.3% on MATH500. Consequently, SimPO is used as the default post-training method in the subsequent experiments.

Performance of Response Simplification Methods As anticipated, the First-Correction Solutions strategy achieves the greatest reduction in length. However, this method decreases performance on the difficult MATH500 test set, which may require more rounds of reflection. The "FCS+Reflection" approach alleviates this issue and surpasses the FCS method by 1.4% with an additional round of reflection. The "Greedily Diverse Solutions" strategy balances performance with the number of generated tokens. However, it significantly underperforms compared to "FCS+Reflection",

reinforcing our claim that the difficult MATH500 test set requires the deep inference provided by o1-like models. Hence, we adopt "FCS+Reflection" as the default response simplification method.

Results on Challenging Test Sets Our approach enhances performance on easier testsets such as ASDIV and GSM8K with fewer tokens, demonstrating the effectiveness and versatility of our method in addressing overthinking issues. To address the concerns of some researchers that our approach might weaken the ability of o1-like models to tackle complex problems requiring long-term reasoning, we validate our method using more challenging GPQA and AIME test sets. As seen, our approach maintains model performance while using fewer tokens, demonstrating the robustness and generalization capability of our approach.

4 Related Work

4.1 Scaling Test-Time Compute

Enhancing model performance on complex tasks can be achieved by scaling test-time compute, which involves:

Expanding Search Space LLMs have strong reasoning abilities, but their auto-regressive decoding often misses optimal solutions. Self-consistency generates multiple responses and use majority voting to select the best answer (Wang et al., 2023b). Other approaches include best-of-n decoding, minimum Bayes risk decoding (Lightman et al., 2024; Li et al., 2023; Khanov et al., 2024; Heineman et al., 2024; Wu et al., 2024), and structured search methods such as Tree-of-Thought, Graph-of-Thought, and Monte Carlo Tree Search (Yao et al., 2024; Besta et al., 2024; Luo et al., 2024; Tian et al., 2024; Wan et al., 2024).

Human-Like Thinking Patterns LLMs often use natural language reasoning. Techniques like chain-of-thought encourage step-by-step reasoning instead of direct answers (Wei et al., 2022b; Kojima et al., 2022). This has been expanded with methods like debating, self-correction, self-critique, and plan-and-solve (Liang et al., 2024; Du et al., 2024; Xiong et al., 2023; Kumar et al., 2024; Kamoi et al., 2024; Ke et al., 2023; Lin et al., 2024; Yu et al., 2024; Wang et al., 2023a). Recent studies also explore latent space reasoning to mimic human cognition (Hao et al., 2024; Goyal et al., 2024). Advanced models combine these patterns into extensive chains-of-thought, improving accuracy with more reasoning time (OpenAI, 2024).

4.2 Efficient Thinking

Scaling the search space and scaling human-like thinking involves two distinct aspects of efficiency: efficient search and efficient thinking. However, few works specifically focus on efficient thinking in LLMs. Kimi et al. (2025) leveraged the long to short strategy to compress generation context. Zhao et al. (2024) encourages the model to terminate reasoning by saying "I don't know" when the problem is hard to solve. Han et al. (2024) introduces token-budget-aware reasoning, where the model is prompted with a specified token budget to guide its reasoning process. There are also several contributions made to predict the distribution of the computation budget and allocate the computation power based on the prompt's difficulty (Damani et al., 2024; Wang et al., 2024; Xu et al., 2024). Another line of work emphasizes the early stopping strategy to save computation budget while reasoning (Manvi et al., 2024; Li et al., 2024). Moreover, multi-agent framework utilizes large LLMs for difficult tasks while small LLMs for simple tasks (Kirchner et al., 2024; Damani et al., 2024)

In summary, all the aforementioned works consider conventional models rather than o1-like models with longer chains-of-thought. In contrast, our work first identifies the overthinking problem in o1-like model. Additionally, instead of limiting the reasoning space or leaving the token budget to be specified by the user, we aim to train the model to learn how to think efficiently.

5 Conclusion

This study identifies a key challenge in o1-like LLMs —- efficient and intelligent scaling of test-time computational resources. We have presented a comprehensive analysis of the overthinking issue in o1-like LLMs. By highlighting the overthinking phenomenon and proposing efficiency metrics, we enhance our understanding of resource utilization in o1-like models. Our self-training based approach effectively mitigates overthinking, reducing unnecessary computation while maintaining performance across reasoning benchmarks of varying difficulty levels.

This work not only improves model efficiency but also sets the groundwork for future research on optimizing computational resource allocation in AI reasoning tasks. Future directions include exploring adaptive compute strategies that dynamically adjust to problem complexity and refining efficiency metrics for broader model generalization.

References

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 17682–17690, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021.

Mehul Damani, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas. Learning how hard to think: Input-adaptive allocation of lm computation, 2024. URL https://arxiv.org/abs/2410.04707.

DeepSeek. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. 2025. URL https://api.semanticscholar.org/CorpusID:275789950.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*, 2024.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=ph04CRkPdC.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Tingxu Han, Chunrong Fang, Shiyu Zhao, Shiqing Ma, Zhenyu Chen, and Zhenting Wang. Token-budget-aware llm reasoning. *arXiv preprint arXiv:2412.18547*, 2024.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space, 2024. URL https://arxiv.org/abs/2412.06769.

David Heineman, Yao Dou, and Wei Xu. Improving minimum bayes risk decoding with multi-prompt. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 22525–22545, 2024.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

Namgyu Ho, Laura Schmid, and Se-Young Yun. Large language models are reasoning teachers. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14852–14882, 2023.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics*, 12:1417–1440, 2024.

Pei Ke, Bosi Wen, Zhuoer Feng, Xiao Liu, Xuanyu Lei, Jiale Cheng, Shengyuan Wang, Aohan Zeng, Yuxiao Dong, Hongning Wang, et al. Critiquellm: Scaling llm-as-critic for effective and explainable evaluation of large language model generation. corr, abs/2311.18702. detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 9004–9017, 2023.

Maxim Khanov, Jirayu Burapacheep, and Yixuan Li. Args: Alignment as reward-guided search. In *The Twelfth International Conference on Learning Representations*, 2024.

Team Kimi, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.

Jan Hendrik Kirchner, Yining Chen, Harri Edwards, Jan Leike, Nat McAleese, and Yuri Burda. Prover-verifier games improve legibility of llm outputs, 2024. URL https://arxiv.org/abs/2407.13692.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213, 2022.

Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, et al. Training language models to self-correct via reinforcement learning. *arXiv preprint arXiv:2409.12917*, 2024.

Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. Making language models better reasoners with step-aware verifier. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5315–5333, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.291. URL https://aclanthology.org/2023.acl-long.291.

Yiwei Li, Peiwen Yuan, Shaoxiong Feng, Boyuan Pan, Xinglin Wang, Bin Sun, Heda Wang, and Kan Li. Escape sky-high cost: Early-stopping self-consistency for multi-step reasoning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=ndR8Ytrzhh.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 17889–17904, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.992. URL https://aclanthology.org/2024.emnlp-main.992.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=v8L0pN6EOi.

Zicheng Lin, Zhibin Gou, Tian Liang, Ruilin Luo, Haowei Liu, and Yujiu Yang. CriticBench: Benchmarking LLMs for critique-correct reasoning. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 1552–1587,

Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.91. URL https://aclanthology.org/2024.findings-acl.91.

Zhihan Liu, Miao Lu, Shenao Zhang, Boyi Liu, Hongyi Guo, Yingxiang Yang, Jose Blanchet, and Zhaoran Wang. Provably mitigating overoptimization in rlhf: Your sft loss is implicitly an adversarial regularizer. *arXiv preprint arXiv:2405.16436*, 2024.

Liangchen Luo, Yinxiao Liu, Rosanne Liu, Samrat Phatale, Harsh Lara, Yunxuan Li, Lei Shu, Yun Zhu, Lei Meng, Jiao Sun, et al. Improve mathematical reasoning in language models by automated process supervision. *arXiv preprint arXiv:2406.06592*, 2024.

Rohin Manvi, Anikait Singh, and Stefano Ermon. Adaptive inference-time compute: Llms can predict if they can do better, even mid-generation, 2024. URL https://arxiv.org/abs/2410.02725.

Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Shen-Yun Miao, Chao-Chun Liang, and Keh-Yih Su. A diverse corpus for evaluating and developing english math word problem solvers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

OpenAI. Learning to reason with llms. https://openai.com/index/learning-to-reason-with-llms, 2024.

Richard Yuanzhe Pang, Weizhe Yuan, He He, Kyunghyun Cho, Sainbayar Sukhbaatar, and Jason E Weston. Iterative reasoning preference optimization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=4XIKfvNYvx.

Qwen. Qwq: Reflect deeply on the boundaries of the unknown, November 2024. URL https://qwenlm.github.io/blog/qwq-32b-preview/.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

Ye Tian, Baolin Peng, Linfeng Song, Lifeng Jin, Dian Yu, Haitao Mi, and Dong Yu. Toward self-improvement of llms via imagination, searching, and criticizing. *arXiv preprint arXiv:2404.12253*, 2024.

Ziyu Wan, Xidong Feng, Muning Wen, Stephen Marcus McAleer, Ying Wen, Weinan Zhang, and Jun Wang. Alphazero-like tree-search can guide large language model decoding and training. In *Forty-first International Conference on Machine Learning*, 2024.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2609–2634, Toronto, Canada, July 2023a. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.147. URL https://aclanthology.org/2023.acl-long.147.

Xinglin Wang, Shaoxiong Feng, Yiwei Li, Peiwen Yuan, Yueqi Zhang, Boyuan Pan, Heda Wang, Yao Hu, and Kan Li. Make every penny count: Difficulty-adaptive self-consistency for cost-efficient reasoning, 2024. URL https://arxiv.org/abs/2408.13457.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023b. URL https://openreview.net/forum?id=1PL1NIMMrw.

Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022a.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022b.

Ian Wu, Patrick Fernandes, Amanda Bertsch, Seungone Kim, Sina Pakazad, and Graham Neubig. Better instruction-following through minimum bayes risk. *arXiv preprint arXiv:2410.02902*, 2024.

Kai Xiong, Xiao Ding, Yixin Cao, Ting Liu, and Bing Qin. Examining inter-consistency of large language models collaboration: An in-depth analysis via debate. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 7572–7590, 2023.

Mayi Xu, Yongqi Li, Ke Sun, and Tieyun Qian. Adaption-of-thought: Learning question difficulty improves large language models for reasoning. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 5468–5495, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.313. URL https://aclanthology.org/2024.emnlp-main.313/.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.

Junyi Ye, Jingyi Gu, Xinyun Zhao, Wenpeng Yin, and Guiling Wang. Assessing the creativity of llms in proposing novel solutions to mathematical problems. *arXiv preprint arXiv:2410.18336*, 2024.

Yue Yu, Zhengxing Chen, Aston Zhang, Liang Tan, Chenguang Zhu, Richard Yuanzhe Pang, Yundi Qian, Xuewei Wang, Suchin Gururangan, Chao Zhang, et al. Self-generated critiques boost reward modeling for language models. *arXiv preprint arXiv:2411.16646*, 2024.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.

Zirui Zhao, Hanze Dong, Amrita Saha, Caiming Xiong, and Doyen Sahoo. Automatic curriculum expert iteration for reliable llm reasoning, 2024. URL https://arxiv.org/abs/2410.07627.

A Appendix

A.1 Case Overview for Deepseek-R1-Preview

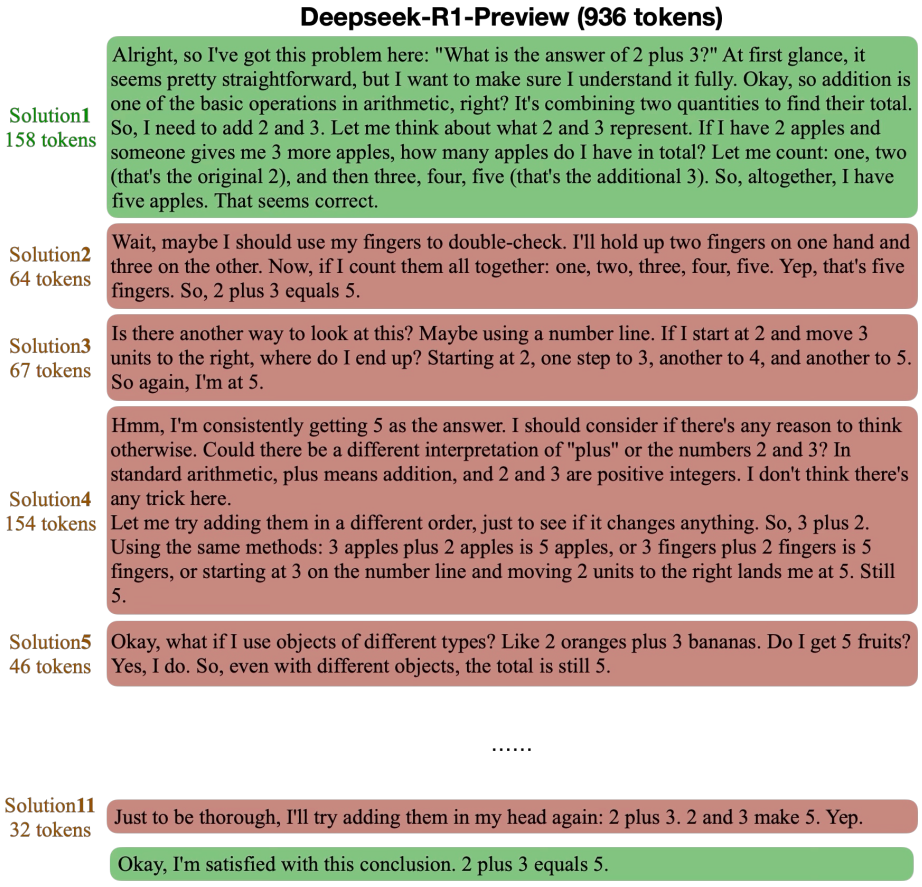


Figure 8: Deepseek-R1-Preview response for the query "What is the answer of 2 plus 3?"

A.2 Prompts for Clustering Solutions

Inspired by (Ye et al., 2024), we leverage GPT-4o to cluster the solutions for each instance into groups with the following prompt:

Criteria for clustering the mathematical solutions:
1. If the solutions used to arrive at the solutions are fundamentally different from each other, such as algebraic manipulation versus geometric reasoning, they can be considered novel;
2. Even if the results are the same, if the intermediate steps or processes involved in reaching those solutions vary significantly, the solutions can be considered different;
3. If the solutions relies on different assumptions or conditions, they should be considered different from each other;
4. A solution might generalize to a broader class of problems, while another solution might be specific to certain conditions. In such cases, they are considered distinct;
5. If one solution is significantly simpler or more complex than the others, it can be regarded as essentially novel, even if they lead to the same result.

Given the following mathematical problem:
problem

Solutions:
Solution 1: ...
Solution 2: ...

Please output the clusters strictly following the following format, each row containing a cluster, names, and reasons. Do not include any additional text or explanations outside of this format:
cluster1 [solution names] reason for cluster
cluster2 [solution names] reason for cluster
cluster3 [solution names] reason for cluster
...

For example:
cluster1 [Solution 1, Solution 3, Solution 5] similar algebraic approach using the volume formula and canceling terms or directly solving for the height.
cluster2 [Solution 2, Solution 4] verifying the correctness and consistency of the formula and solution and considering unit checks or logical reasoning on how volume relates to base area and height.